Memories, not Packets: A Zero-Dependency `no_std` Memory Substrate for AI Agents with Conflict-Preserving Peer Synchronization

Marcelo Scapin Rovani*
`neural-os-core` community extraction (MIT OR Apache-2.0)

August 11, 2026

Abstract

AI agents operate with ephemeral context windows: memory is lost between turns, across instances, and after crashes. We present neural-sgdb, a persistent, transferable memory database for AI agents extracted from a bare-metal operating system, designed under strict constraints: *zero external dependencies*, dual `no_std` + `std` operation, and byte-identical on-disk formats shared with the parent OS. The system stores *memories*—documents with a cognitive layer (L_0–L_7), a vector clock and an identity—rather than opaque data. Semantic recall combines binary quantization (BQ) as a coarse candidate filter with FP32 cosine rescore, SIMD Hamming dispatch (AVX-512/AVX2/scalar), a multi-index-hashing (MIH) accelerator, and an optional lexical BM25-style dual path. Storage is an append-log with per-record CRC, atomic compaction, and a byte-exact log format (TKLV/TKCK) with checkpoint fast-mount. Peers synchronize memories through a conflict-preserving CRDT: concurrent writes are *preserved*, never last-writer-wins-discarded, and an AI at the root of the process arbitrates the preserved conflicts at read time using recency, importance and temporal validity. We report measured performance (an order-of-magnitude write throughput improvement via a persistent lazy storage handle, microsecond recall, 3× faster volume mounting under churn), a reproducible two-instance peer-synchronization convergence demo (agent-to-agent memory replication), and the honest costs of the model: eventual consistency, absence of global ordering, and deferred conflict resolution.

Keywords: AI-agent memory; CRDT; conflict-free replicated data types; binary quantization; approximate nearest neighbor; `no_std`; adaptive radix tree; cognitive architectures

1 Introduction

State-of-the-art AI agents are bounded by a context window: reasoning state, tool results and conversation are ephemeral and do not survive the session. Recent systems address this with external memory—agentic file stores, retrieval-augmented generation over long-term stores, and hierarchical cognitive memories [1, 2, 3, 4, 5]. Most of these assume a central database, a network stack, and a dependency-rich runtime.

We take a different premise, inherited from a bare-metal operating system that boots with AI: memory should be a first-class substrate that lives on the device, survives power loss, transfers between instances, and does not require a server. This leads to a design space rarely explored together: *zero dependencies*, ***`no_std` + `std` dual mode***, *format stability across an OS boundary*, and *peer-to-peer memory exchange with conflict preservation*.

The contribution of this paper is the neural-sgdb system and the empirical evidence around it:

*Project: https://github.com/msrovani/neural-sgdb

1. A memory model with 8 cognitive layers, a length-prefixed binary document format (NMD1) byte-identical to the parent OS, and a vector clock with semantic equality.

2. A recall stack combining BQ coarse filtering, FP32 rescore, SIMD Hamming dispatch, auto-oversampling by dimensionality, MIH acceleration, and an optional lexical dual path—all format-neutral.

3. A storage layer (CRC append-log and a byte-exact log codec) with checkpoint fast-mount, atomic compaction, and in-place tombstone invalidation.

4. A conflict-preserving CRDT synchronization protocol (peer-to-peer memory exchange)[1] with delta sending, and a principled arbitration model in which an AI at the root of the process resolves preserved conflicts at read time.

5. Measured results: $\sim 38\times$ faster writes from a single storage optimization, microsecond recall, $\sim 2.9\times$ faster volume mounting under churn, and a reproducible two-instance convergence experiment.

We are explicit about what we do *not* claim: no linearizability, no global ordering, and no automatic conflict resolution. These are the honest costs of the peer-to-peer model, and we argue they are acceptable—and in some respects preferable—for agent memory.

2 Related Work

Agent memory. MemGPT/Letta [1, 2] formalize hierarchical memory (working vs. archival) with paging and background consolidation; Mem0 [3] adds multi-signal retrieval and distillation; Zep/Graphiti [4] store temporal knowledge graphs with validity windows and provenance; Anthropic's contextual retrieval [5] demonstrates large retrieval-failure reductions by prepending chunk context and adding a lexical (BM25) path and a reranker. We share the memory-as-substrate view and adopt the lexical dual path and validity-window patterns, but keep zero dependencies and peer-to-peer sync.

Approximate nearest neighbor. Sign-based binary quantization is a compact candidate generator in production ANN systems [6]; Qdrant reports high recall with 2–4× oversampling followed by full-precision rescore. Multi-index hashing [7] turns exact Hamming search over binary codes into a sub-linear candidate search. ScaNN [8] and Weaviate [9] explore anisotropic and multi-codebook quantization. We adopt oversampling and MIH; multi-codebook quantization is future work that would require a new opt-in encoding (our stored bit-vectors are a format contract).

Log-structured storage. Append-only logs with per-record checksums, checkpointing and compaction [10, 11] are the right family for a single-process substrate; LSM compaction amplification [12] is unnecessary at this scale. Our TKLV/TKCK codec follows this family and is byte-identical to the parent OS.

Conflict-free replicated data types. CRDTs [13] give eventual convergence under commutativity. Delta state CRDTs [14] shrink sync payloads; dotted version vectors [15] compress causal metadata. Our protocol is a version-counter CRDT with explicit conflict preservation (a multi-value register), delta sending, and a caller-supplied clock seam (wall-clock time is never assumed by the CRDT itself).

3 System Design

3.1 Memory model and format

A memory document carries a layer $L \in \{L_0, \ldots, L_7\}$ (sensory, working, episodic short/long, semantic, procedural, reserved, identity), a key, a vector clock, a payload, and an

[1] *Terminology.* We use *peer-to-peer memory synchronization* throughout; the same mechanism is also described—equivalently, in our view—as *memory exchange*, *gossip-style memory dissemination*, *causal memory propagation*, *synchronized memory transfer*, or *agent-to-agent memory replication*. We treat these as synonyms; readers may use whichever they prefer.

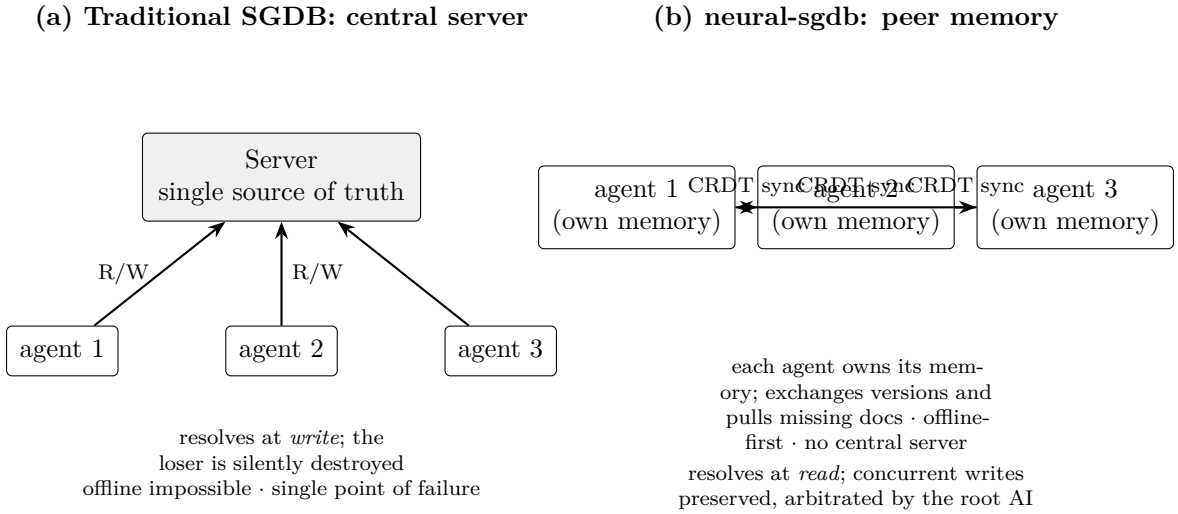

Figure 1: Two memory architectures. (a) Traditional: agents read/write a central server; the server resolves conflicts at write time and silently destroys the loser; agents are unusable offline. (b) neural-sgdb: each agent owns a local memory, peers synchronize causally through a conflict-preserving CRDT, and concurrent versions survive until an arbiter resolves them at read time.

optional packed binary embedding. The NMD1 encoding is length-prefixed and byte-identical to the parent OS; golden byte tests pin the layout. A zero-copy view allows payload access without full deserialization.

Layers L_0/L_1 live in RAM and flush to storage at an explicit checkpoint; L_2–L_7 persist immediately. Semantic memories (L_4/L_5) store the caller-supplied embedding payload and a 1-bit quantized bit-vector $\text{sign}(x) > 0$; the textual companion lives in L_2.

3.2 Recall

Recall follows a Qdrant-style coarse-to-fine pipeline:

$$\mathit{recall}(q,k) = \text{top}_k\big(\text{rescore}_{\text{FP32}}\big(\text{top}_{k\cdot s}(\text{BQ}(q))\big)\big), \quad (1)$$

where the Hamming distance uses SIMD dispatch (AVX-512 > AVX2 > scalar, selected at runtime via `cpu_caps()`) and the oversampling factor s is chosen automatically by dimensionality (1 word $\to$ 16, 2–4 words $\to$ 8, else 4) because 1-bit BQ degrades below $\sim$768 dimensions. A bounded max-heap keeps top-k ranking $O(N \cdot D/64 + N \log k)$.

Two accelerators are format-neutral: *MIH* partitions each stored bit-vector into blocks, indexes block hashes, and probes the query blocks to recover sub-linear candidate sets [7]; and a *lexical* BM25-style inverted index over L_2/L_3 texts implements the dual-path pattern of contextual retrieval [5], merged with semantic hits in `recall_hybrid`. Ranking may be weighted by recency (from a wall-clock timestamp embedded in the key), importance (from the layer), and semantic distance.

3.3 Storage

Backends implement a four-method `Storage` trait. `FileStorage` is a CRC-protected append-log with atomic compaction; `TickvFile` is the byte-exact TKLV codec of the parent OS, with a TKCK checkpoint record enabling *fast-mount* (header-only scan, FNV index verification, per-entry stale check) with a full-scan fallback, and in-place tombstone invalidation (`magic`[3]=0) before overwrite—making dead space header-detectable. A single measured optimization, a persistent lazy append handle, reduced write latency by $\sim 38\times$ (Section 5).

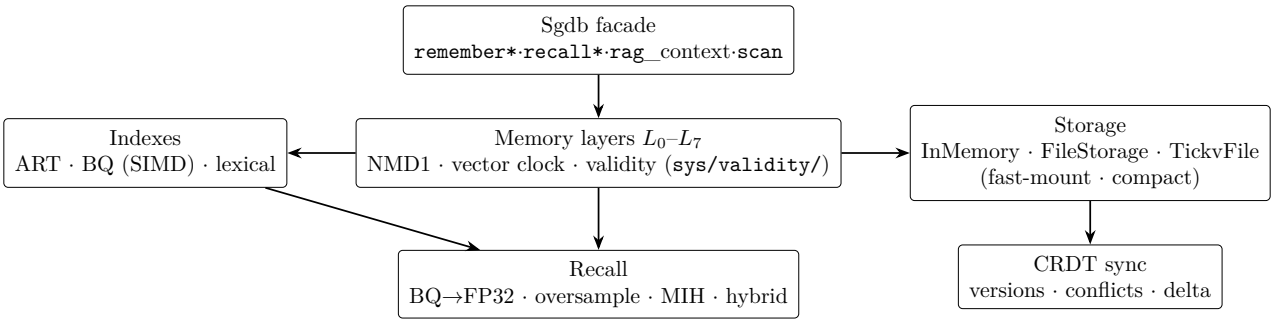

Figure 2: Architecture of neural-sgdb. The Sgdb facade exposes *memories, not data*: documents with a layer, a vector clock and a validity window. Derived indexes (ART, BQ, lexical) and pluggable storage are seams; peer synchronization rides on the storage layer. All components compile to `no_std` (only `alloc`) with zero external dependencies.

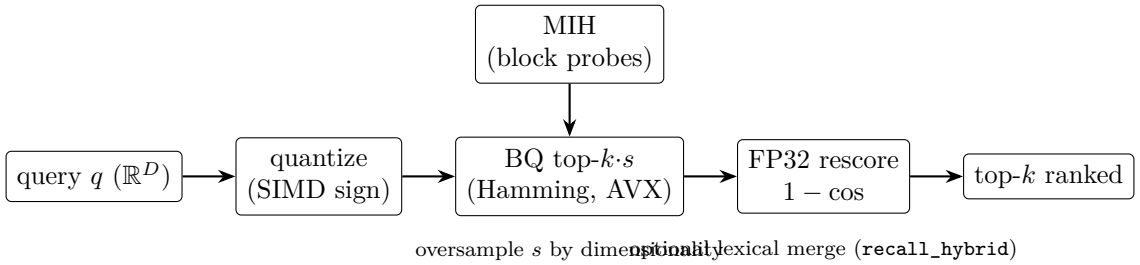

Figure 3: Coarse-to-fine recall (Eq. 1). A coarse binary-quantized Hamming filter (SIMD-dispatched, optionally accelerated by multi-index hashing) retrieves an oversampled candidate pool; full-precision cosine rescore re-ranks it. All stages are format-neutral—stored bit-vectors never change.

3.4 Peer-to-Peer Memory Synchronization

Each node pairs an `Sgdb` instance with a `CrdtMemorySync` holding its own version counter, a count of independent local writes (`own_writes`), the versions known of peers, and a preserved conflict set. The merge is a multi-value register with explicit verdicts: self-packets ignored, stale/duplicate versions rejected (no regression), new versions adopted when no conflicting local state exists, and *concurrent* versions preserved in the conflict set. Delta sending (§3.4) ships only versions a peer has not seen—gossip-style dissemination of the causal state rather than the full history.

Exchanging versions is the causal trigger; transferring documents (memory transfer) is a diff-pull: a peer that learns another advanced replicates the missing docs (`get` → `put`). Two instances converge with no central server. Because resolution writes are themselves causal CRDT writes, arbitration outcomes propagate to every peer via the same dissemination mechanism.

3.5 Arbitration by an AI at the root

Because concurrent writes are preserved rather than resolved, the system defers semantic decisions to the layer that has the full context: an AI at the root of the process (in the parent OS, running from boot). Arbitration is a five-step pipeline: *harvest* the preserved conflict set (complete information is guaranteed—no update was destroyed); *contextualize* along the causal chain using episodic memories; *weigh* recency (wall clock in the key), importance (layer) and behavioral consistency; *decide* via one of four policies—deterministic recency-first ranking at read time, temporal validity invalidation (invalidate-not-delete), semantic fusion (write a unified document and `supersede` the originals),

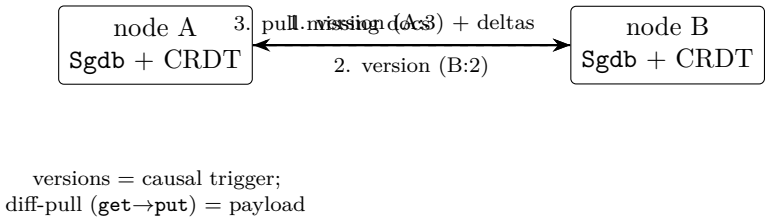

Figure 4: Peer-to-peer memory exchange between two instances. Versions (and deltas) are the causal trigger; the pull phase replicates only the documents a peer is missing. The merge preserves concurrent writes (multi-value) instead of discarding them.

or escalation to a human/agent; and *propagate* the resolution as a new causal write.

This contrasts with the traditional resolve-at-write model, where a central server silently destroys the losing version. Here arbitration runs on complete data, at any time, and the verdict is CRDT state that converges across peers.

4 Design Rationale

Four constraints shape the system, and each has a concrete consequence:

- **Zero external dependencies.** The substrate is meant to run inside an operating system and to be audited line by line. Every algorithm—ART, BQ/Hamming, CRC32, FNV, BM25, CRDT—is implemented in-crate. This is a deliberate cost: no reuse of mature libraries, in exchange for portability and a small trusted-computing base.

- **Dual `no_std` + `std`.** The core is `no_std` (only `alloc`); storage backends and the UDP demo transport are `std`-gated features. The same code runs on bare metal and on the host, enforced by a CI gate (`cargo check --no-default-features --target x86_64-unknown-none`).

- **Byte-identical formats.** NMD1 (documents) and TKLV/TKCK (storage) are contracts shared with the parent OS: golden byte tests pin the layout, and a volume written by either side mounts on the other. Format changes therefore require synchronized change on both sides—new capabilities are introduced as new opt-in encodings or side indices, never by mutating existing bytes.

- **Seams, not globals.** There is no global engine: the clock is a `now: u64` parameter, SIMD capability is injected via `cpu_caps()`/`set_cpu_caps()`, and storage is the four-method `Storage` trait. This keeps the substrate testable and embeddable in a kernel.

We also deliberately chose an append-log with checkpoints over an LSM [12]: at this scale (thousands to millions of small memories on a single device), write/read amplification from compaction is pure cost, while a CRC-protected append-log with an atomic compact and a TKCK fast-mount gives power-loss safety with a fraction of the machinery.

5 Evaluation

All measurements are from the project's benchmark/stress harnesses on a consumer laptop (AVX2; no AVX-512). The test suite is 92+1 unit/doc tests (default features) and 102+1 with the `p2p` feature; the `no_std` gate (`cargo check --no-default-features --target x86_64-unknown-none`) is clean.

5.1 Storage write path

Replacing a per-write file open/close with a persistent lazy append handle reduced raw `put` latency from 185 μs to 4.8 μs ($\sim 38\times$), and end-to-end semantic writes from 422 μs to 22 μs ($\sim 19\times$).

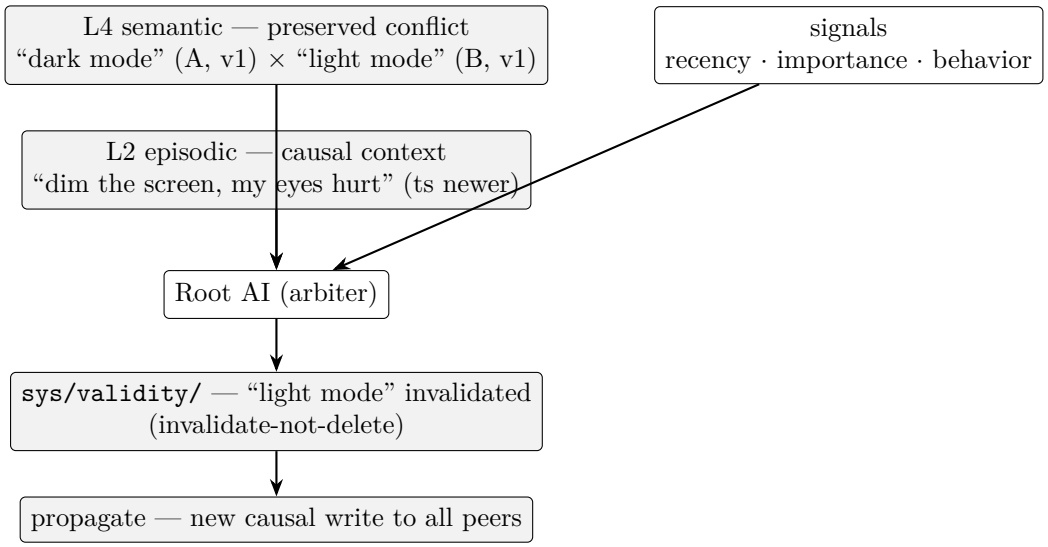

Figure 5: Layered arbitration of a preserved conflict (§3.5). The root AI combines the two concurrent L4 versions with the causal L2 context and a recency/importance/behavior signal, then materializes the verdict as a validity window—an invalidation that is itself a new causal write and propagates to every peer.

A crash-invariant test verifies that writes after atomic compaction target the new file (never a stale inode/file object).

5.2 Recall

BQ top-5 over 10 000 vectors × 1024 dimensions dropped from 213 μs to 160 μs/query after removing a locked atomic RMW from the per-call Hamming dispatch ($\sim$ 25%). On a correlated-cluster corpus, the coarse BQ filter alone recovers 22% of the exact FP32 top-5 at oversample 1×, rising to 35% at 16×—consistent with the known property that sign-BQ separates the *cluster* but not the *exact member*; the FP32 rescore then re-ranks the candidates. With 16-dimension embeddings, low-dimensional bit collision drops exact@1 to $\sim$ 42%; raising the candidate pool via oversampling restores it.

5.3 Fast-mount under churn

On a churned volume (35 000 records, 5 000 live), checkpoint fast-mount took 14.8 ms vs. 43.2 ms for a full scan ($\sim$ 2.9×). On an all-live volume both read the same bytes and are at parity; the win is not re-processing tombstones/dead records.

5.4 Peer Synchronization Convergence

The two-instance synchronization experiment (the `p2p_telepathy` example) runs two `Sgdb` instances over an in-memory transport: A writes m_1, B writes m_2; one synchronization round replicates two documents in each direction and logs both concurrent writes as preserved conflicts; a second round is idempotent; a reply m_3 propagates back in a third round. Both instances converge to identical memory sets and cross-recall each other's memories semantically. Convergence is idempotent and requires no central coordinator.

6 Discussion and Limitations

Eventual consistency. Peers diverge during the window between a write and a synchronization round. Convergence is guaranteed (the merge is commutative, associative, idempotent, and monotonic) but not time-bounded. For agent memory this is a feature—an agent's memory is local and survives disconnection—but applications requiring linearizable reads must add their own coordination.

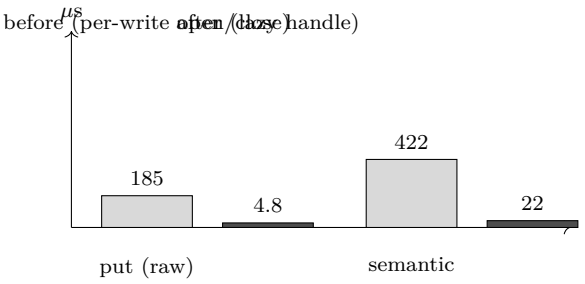

Figure 6: Write latency before and after the persistent lazy append handle: raw `put` 185 → 4.8 μs ($\sim$ 38×) and end-to-end semantic writes 422 → 22 μs ($\sim$ 19×). Bars are to the same scale; values annotated.

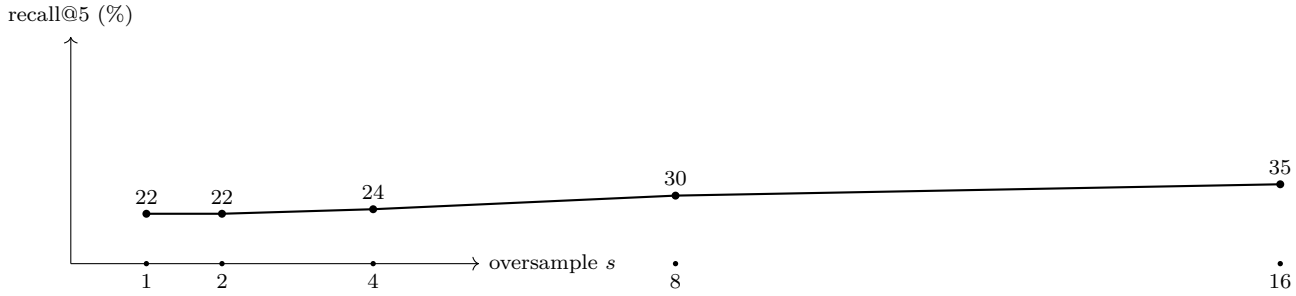

Figure 7: Coarse BQ candidate recall@5 vs. oversample on a correlated 1024-dimension cluster corpus. The coarse filter separates the cluster but not the exact member; enlarging the candidate pool raises recall (22% at 1× to 35% at 16×), and the FP32 rescore then re-ranks the candidates.

No global ordering. There is no global sequence; the CRDT provides causal order only, and wall-clock time must be supplied by the caller. "Latest" queries therefore require a caller clock (as in our weighted ranking).

Deferred conflict resolution. Conflicts are preserved, not resolved, by design. Resolving them requires a policy in the consuming layer; we provide primitives (weighted ranking, validity windows, supersede) and an arbitration model, but the semantic judgment is not automated by the substrate.

Other limitations. The demo transport is unauthenticated (production requires a signed transport); the system is single-threaded by design; ARM SIMD (NEON) and multi-codebook quantization are future work; the MCP embedding is a demo trigram hash, not a real semantic model.

6.1 Threats to validity

Measurement. All numbers come from the project's benchmark harness on a single consumer laptop (AVX2, no AVX-512), not a controlled experimental protocol. Relative comparisons (before/after) are robust; absolute figures may not transfer.

Embeddings. Synthetic embeddings (LCG, trigram hash, cluster+noise) approximate, but do not reproduce, production model embeddings; the BQ trade-off curve may shift with real distributions.

Networking. The convergence experiment runs over an in-memory transport; real networks (ordering, loss, latency) are not exercised.

Concurrency. The substrate is single-threaded by design; no multi-threaded throughput or consistency results are reported.

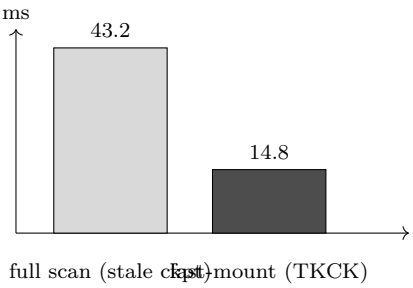

Figure 8: Volume mount on a churned store (35 000 records, 5 000 live): full scan 43.2 ms vs. TKCK fast-mount 14.8 ms ($\sim 2.9\times$). The checkpoint index skips tombstones/dead records.

7 A Case Study: an Agent Memory Lifecycle

We sketch one complete agent scenario to tie the pieces together.

Turn. The agent answers a user query; `remember_exchange` stores the exchange in L_1/L_2 (RAM), and `remember_semantic` indexes the summary in L_4 with a caller-supplied embedding.

Recall. On the next query, the embedding is run through the coarse-to-fine pipeline (Eq. 1): auto-oversampling widens the BQ pool, `recall_weighted` re-ranks by recency (from the `ts` key), importance (layer) and semantic distance.

Consolidation (future). A background pass distills episodic L_2 into a semantic L_4 fact, written as a new causal document—the substrate "dreams" without dropping the episode.

Transfer. A second instance of the agent meets the first; the peer-synchronization protocol (§3.4) exchanges versions and pulls missing memories. Both converge; neither needed a server.

Arbitration. The two instances had concurrently recorded different preferences; the conflict is preserved. The root AI (Fig. 5) combines the L4 conflict with the L2 causal context and a newer timestamp, and invalidates the stale value via `sys/validity/`—an invalidate-not-delete that propagates to every peer.

Forgetting. A stale fact whose validity window has passed no longer surfaces in `recall_at`, while its history remains auditably intact.

8 Reproducibility

Source, tests and harnesses are public: https://github.com/msrovani/neural-sgdb. Key commands: `cargo test` (unit), `cargo test --features p2p`, `cargo check --no-default-features --target x86_64-unknown-none` (`no_std` gate), `cargo run --release --example bench`, `cargo run --release --example stress`, and `cargo run --release --example p2p_telepathy --features p2p` (two-instance synchronization experiment).

9 Future Work

Planned directions include background consolidation ("dreaming") that distills episodic memories into semantic facts as new causal writes; arbitration over validity windows in the sync layer; a signed/authenticated transport; NEON SIMD; and multi-codebook quantization as an opt-in encoding that preserves existing stored vectors.

References

[1] C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, J. Gonzalez. "MemGPT: Towards LLMs as Operating Systems," *arXiv:2310.08560*, 2023.

[2] Letta project. "Letta: Stateful agents framework." https://docs.letta.com.

[3] Mem0 project. "The memory layer for personalized AI." https://mem0.ai.

[4] Zep/Graphiti. "Zep: Temporal Knowledge Graph Architecture for Agent Memory," *arXiv:2501.13956*, 2025.

[5] Anthropic. "Introducing Contextual Retrieval," 2024. https://www.anthropic.com/news/contextual-retrieval.

[6] Qdrant. "Binary Quantization." https://qdrant.tech.

[7] M. Norouzi, A. Punjani, D. Fleet. "Fast Exact Search in Hamming Space with Multi-Index Hashing," *IEEE TPAMI* 36(6), 2014.

[8] R. Guo et al. "Accelerating Large-Scale Inference with Anisotropic Vector Quantization," *ICML*, 2020.

[9] Weaviate. "Vector index: HFresh/SPFresh and dynamic *ef*.", https://weaviate.io.

[10] Basho. "Bitcask: A Log-Structured Hash Table for Fast Key/Value Data," 2010.

[11] SQLite. "Write-Ahead Logging." https://sqlite.org/wal.html.

[12] RocksDB. "Overview of Architecture." https://github.com/facebook/rocksdb.

[13] M. Shapiro et al. "A Comprehensive Study of Convergent and Commutative Replicated Data Types," *RR-7506, INRIA*, 2011.

[14] C. Almeida, A. Shoker, C. Baquero. "Delta State Replicated Data Types," *arXiv:1603.01529*, 2016.

[15] N. Preguiça et al. "Dotted Version Vectors: Efficient Causality Tracking for Distributed Systems," *OPODIS*, 2010.

[16] V. Leis, A. Kemper, T. Neumann. "The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases," *DaMoN*, 2013.

[17] S. Sumers, T. Yao, K. Narasimhan, T. Griffiths. "Cognitive Architectures for Language Agents," *TMLR*, 2024.

A Peer-synchronization experiment trace

The two-instance synchronization experiment (`cargo run --release --example p2p_telepathy --features p2p`) prints the following trace; concurrent writes are reported as preserved conflicts.

```
[A] lembra m1  (versão A = 1)
[B] lembra m2  (versão B = 1)
CRDT sync: node=2 v=1 CONFLITO preservado (concorrente)
CRDT sync: node=1 v=1 CONFLITO preservado (concorrente)
[ ] ronda 1: A→B 2 doc(s), B→A 2 doc(s)
[ ] ronda 2: A→B 0 doc(s)
[B] lembra m3  (versão B = 2)
[ ] ronda 3: A→B 0 doc(s), B→A 2 doc(s)
[ ] A conhece 6 docs, B conhece 6 docs
[B] recall da memória de A: ["eu sou a instancia A..."]
```

Both instances converge to the same memory set (three memories × L2 text + L4 embedding = six documents) and cross-recall each other's memories semantically. Round 2 is idempotent. The resolution of a later conflict (a new causal write, e.g. a validity invalidation) propagates through the same mechanism.